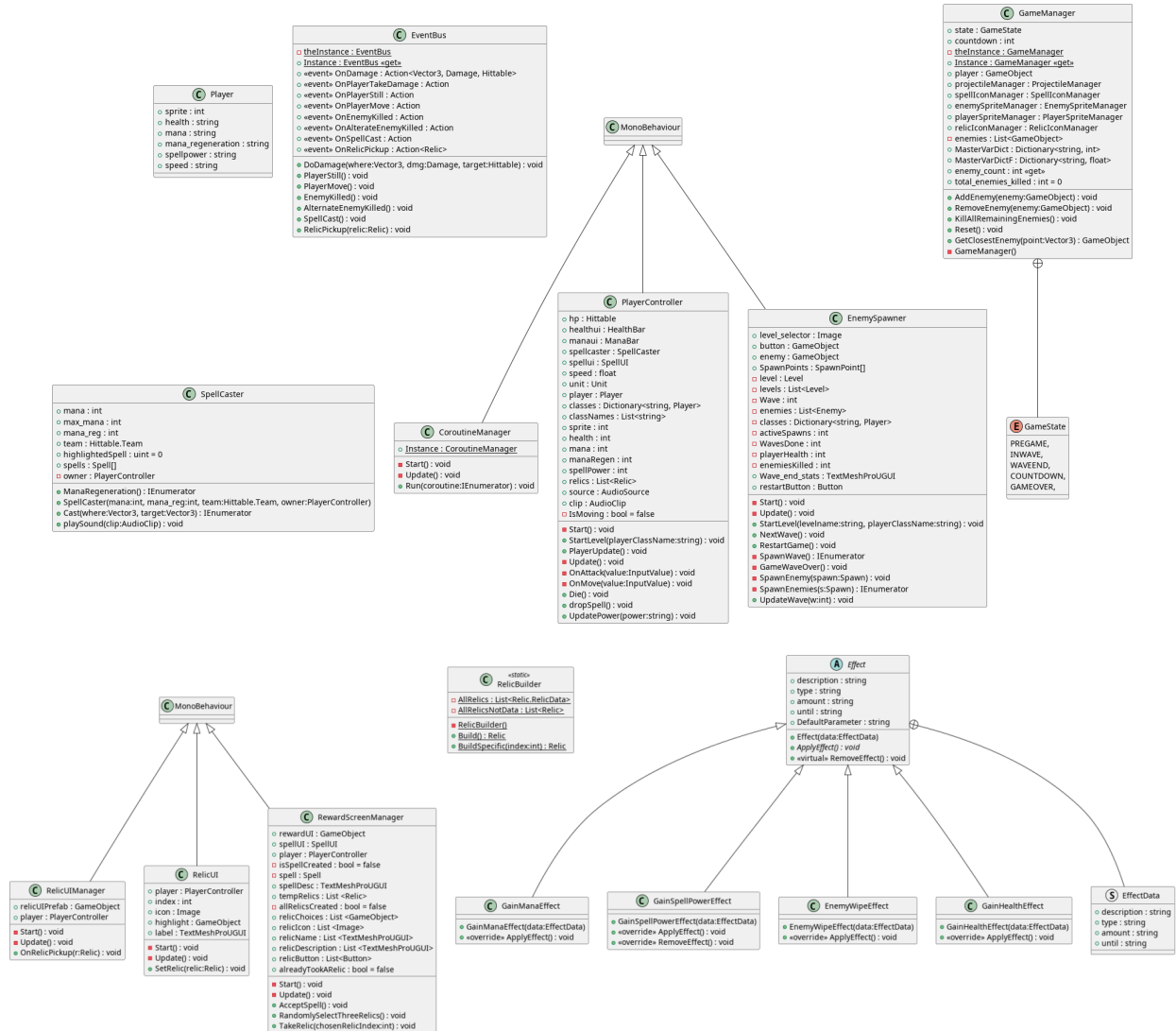
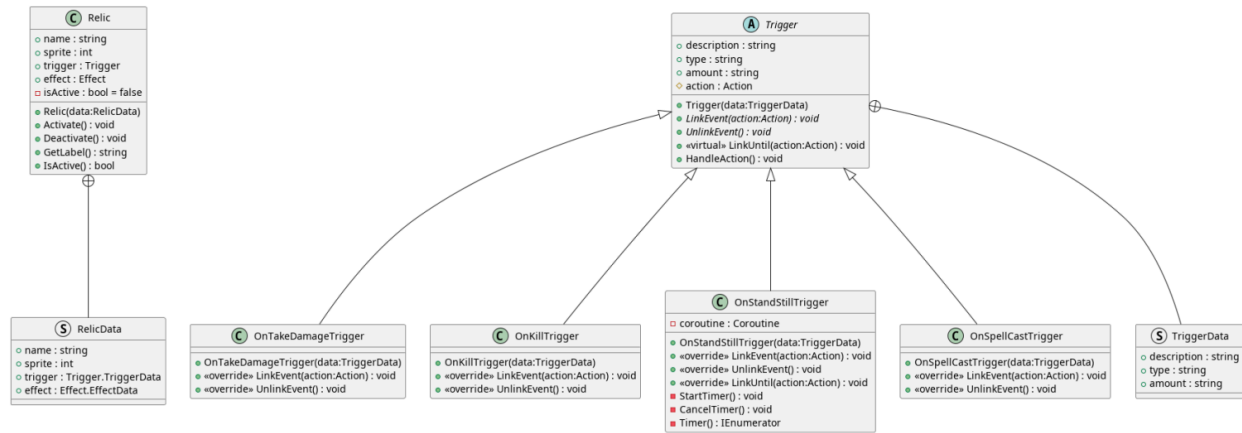


Report.pdf for Project 4: Relics

Partners: Claire Buck & Calvin Richards

Architecture Diagram





Textual Description of Architecture

RelicBuilder reads the json file, sends information to Relic to create Relic objects, and “Builds” relics, which basically means it picks out a relic from the possible relics list that the player doesn’t already own. Relic sets up relic objects, which have names, sprites, a Trigger object and an Effect object. It also has Activate() and Deactivate() methods, which basically tell the game when to or not to listen for Triggers and make Effects happen. Triggers talk to the EventBus to find the proper event to trigger them. The Relic constructor then connects triggers to effects. In RewardScreenManager, the function OnRelicPickup is called, which makes every relic that the player has display the photo of the relic on the bar on the top so the player can visually see what they have in-game. RelicUI is kind of not used at all, because I (Claire) did not understand how to use it the way it was set up by default, and all of the stuff to display Relics is in either RewardScreenManager or RelicUIManager.

Calvin’s Paragraph

I did a lot of the architecting of the relics system: created the Effect.cs , Trigger.cs , and [Relic.cs](#) files alongside [RelicBuilder.cs](#) . I created most of the events in the EventBus and hooked them up. I had to do some research when creating the triggers for effects that used an “until” parameter (figuring out a timer coroutine). I cleaned up a lot of the software architecture: creating a universal dictionary to be used with the RPNEvaluator, and generally cleaning things up. I added the player class selection screen and updated the health to scale correctly. It was nice to feel like I knew what I was doing from the beginning, although with the player stop/start trigger that took a long time to solve. Things kind of grinded to a halt as we got closer to finishing, but we always made steady progress. As the project has gone on it has become more and more important to structure things well and ensure that each system communicates with each other in an efficient and sensible manner.

Claire's Paragraph

I worked mainly in RelicBuilder, RewardScreen, and RelicUI. I also wrote our unique relics with their triggers and effects, but Calvin helped implement them. A lot of my time was spent trying to figure out how to use RelicUI, and how to select 3 random relics (with no dupes), and display them on the Reward Screen. I think this was the most challenging, but most rewarding, part of any of the projects so far. I was in the Unity editor a lot more, and I was able to figure out how to connect up stuff from the Unity editor to my files without much outside help. I feel like I actually understand a lot more about the Unity editor than I did before, especially because another large chunk of my time was spent editing the Reward Screen layout; it was really messy and jumbled before, and it was a lot of fiddling to get it looking better. I also wrote some of the functions in [Relic.cs](#), and I think that helped me by familiarizing me more with triggers and events. Honestly, I still find them a bit confusing, and all of the connections can be hard for me to follow at times, but I think since Relic seems to work ok, it's a good starting point.